



FULL-STACK APPLICATION DESIGN AND DEVELOPMENT

COURSE 8-9 - DJANGO



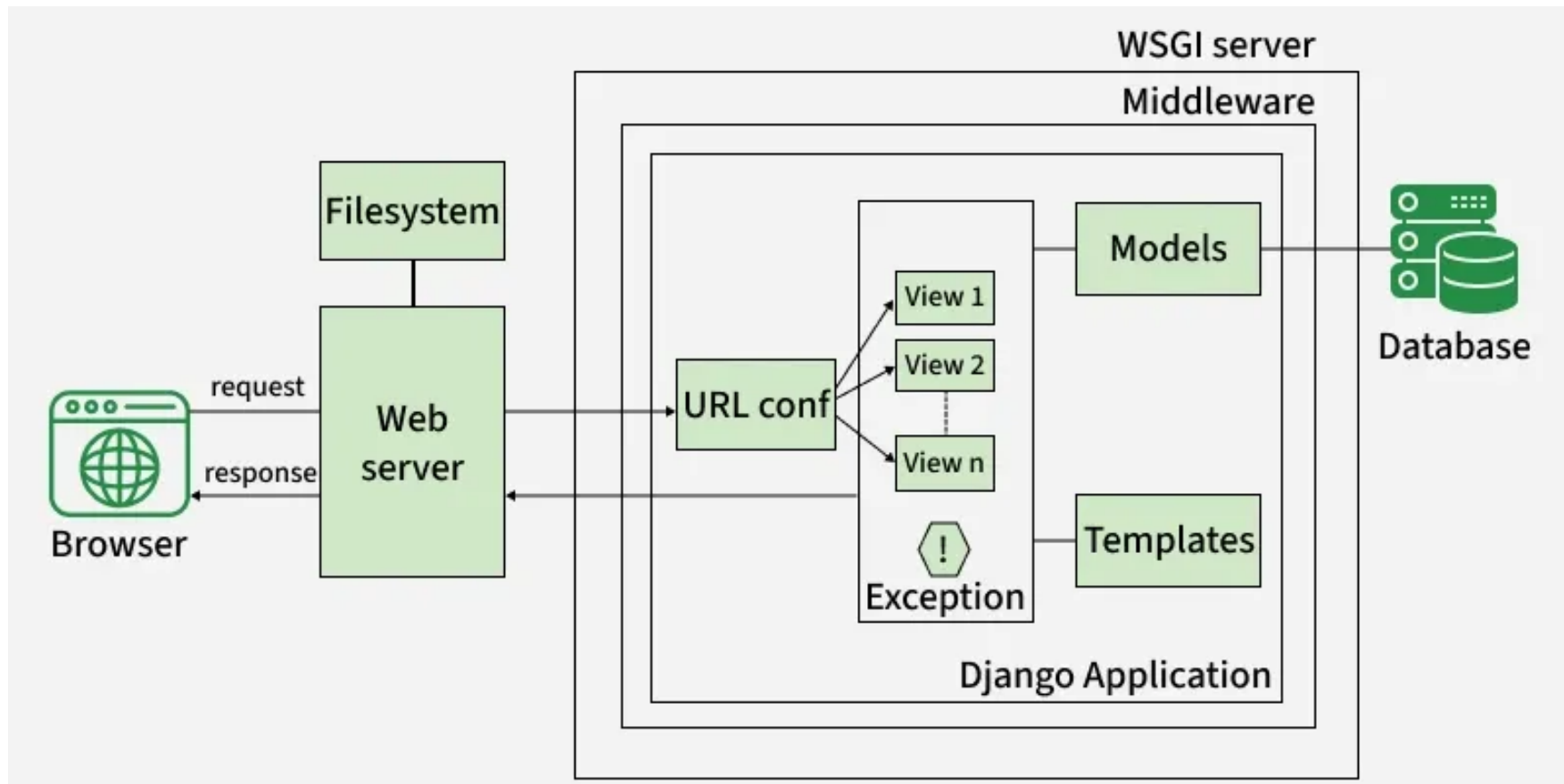
INTRODUCTION

- Django is a **Python-based web framework** that makes it easier to build secure, scalable and maintainable web applications.
- It takes care of tasks like database handling, user authentication, and routing so developers can focus on features instead of base code.
- It comes with many built-in features like user authentication, an admin panel and form handling.
 - **DRY Principle:** It follows the principle of DRY (Don't Repeat Yourself).
 - **Full-Stack Framework:** Manages both backend logic and frontend rendering.
 - **Rapid Development:** Enables quick creation of complete web applications.
 - **Admin Interface:** Auto-generates an admin panel for easy data management.
 - **Database Support:** Works seamlessly with SQLite, MySQL, and PostgreSQL.
 - **Scalable:** Efficiently handles growing traffic and application size.

INTRODUCTION

- Django is based on MVT (Model-View-Template) architecture. MVT is a software design pattern for developing a web application. It's structure has the following three parts:
 1. **Model:** Acts as the data interface. It defines the structure of your data and is usually backed by a database (e.g., MySQL, PostgreSQL).
 2. **View:** A Python function or class that handles web requests. It interacts with the Model and renders a response, typically by passing data to a Template.
 3. **Template:** Contains static HTML mixed with Django's templating syntax. Templates are used to display dynamic data on the web page.

INTRODUCTION



INTRODUCTION

- **Browser Sends Request:** The browser sends a HTTP request to the web server (e.g., typing `http://127.0.0.1:8000/`).
- **Web Server Receives It:** The server (like Nginx or Apache) serves static files directly or forwards other requests to a WSGI/ASGI server such as Gunicorn or Daphne.
- **Request Reaches Django:** The Web Server Gateway Interface (WSGI)/Asynchronous Server Gateway Interface (ASGI) server converts the request into a Python-friendly format and passes it through Django's middleware before it reaches your app.
- **URL Routing:** Django checks `urls.py` to find the correct view function for the request, turning it into a `HttpRequest` object.
- **View Handles Logic:** The view processes the request - querying the database, rendering templates, or returning data (like JSON).
- **Response Created:** The view returns a `HttpResponse` object, which Django converts into an HTTP response.
- **Response Sent to Browser:** The response passes back through the server and is displayed by the browser as a web page.

INTRODUCTION

- **To set up Django in a Linux system follow these steps:**

1. **Install Python:** Download and install the latest Python 3 version from the official website.
2. **Install pip:** Pip comes with recent versions of Python. Open the command prompt and run:

```
pip3 --version
```

3. **Set Up a Virtual Environment:** Create a virtual environment folder named **venv**.

```
python3 -m venv venv
```

4. **Activate the Environment:**

```
source venv/bin/activate
```

5. **Install Django:**

- With your virtual environment active, run:

```
pip install django
```

INTRODUCTION

"Hello world!" in Django

- To display a simple "Hello world!" message in the browser, start by creating a `views.py` file in your app directory. It defines how Django responds to an incoming request.

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("<h1>Hello world!</h1>")
```

- Next, configure `urls.py` to route the incoming URL to this view. The URL configuration maps a specific URL path to the corresponding view function so Django knows what code to execute when a user visits that URL.

In **`urls.py`**:

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.home, name="home"),
]
```

- Visiting the development server at "`http://localhost:8000/`" will display: Hello world!

INTRODUCTION

- Django follows the **MVT (Model-View-Template)** architectural pattern, which is a variation of the traditional MVC (Model-View-Controller) design pattern used in web development.
- This pattern separates the application into three main components:
 - I. **Model**
 - Model acts as the data layer of an application. It defines the structure of the database and handles data-related logic.
 - Models typically represent database tables.
 - Responsible for querying, inserting, updating, and deleting data.
 - Usually backed by relational databases such as MySQL, PostgreSQL, or SQLite.

INTRODUCTION

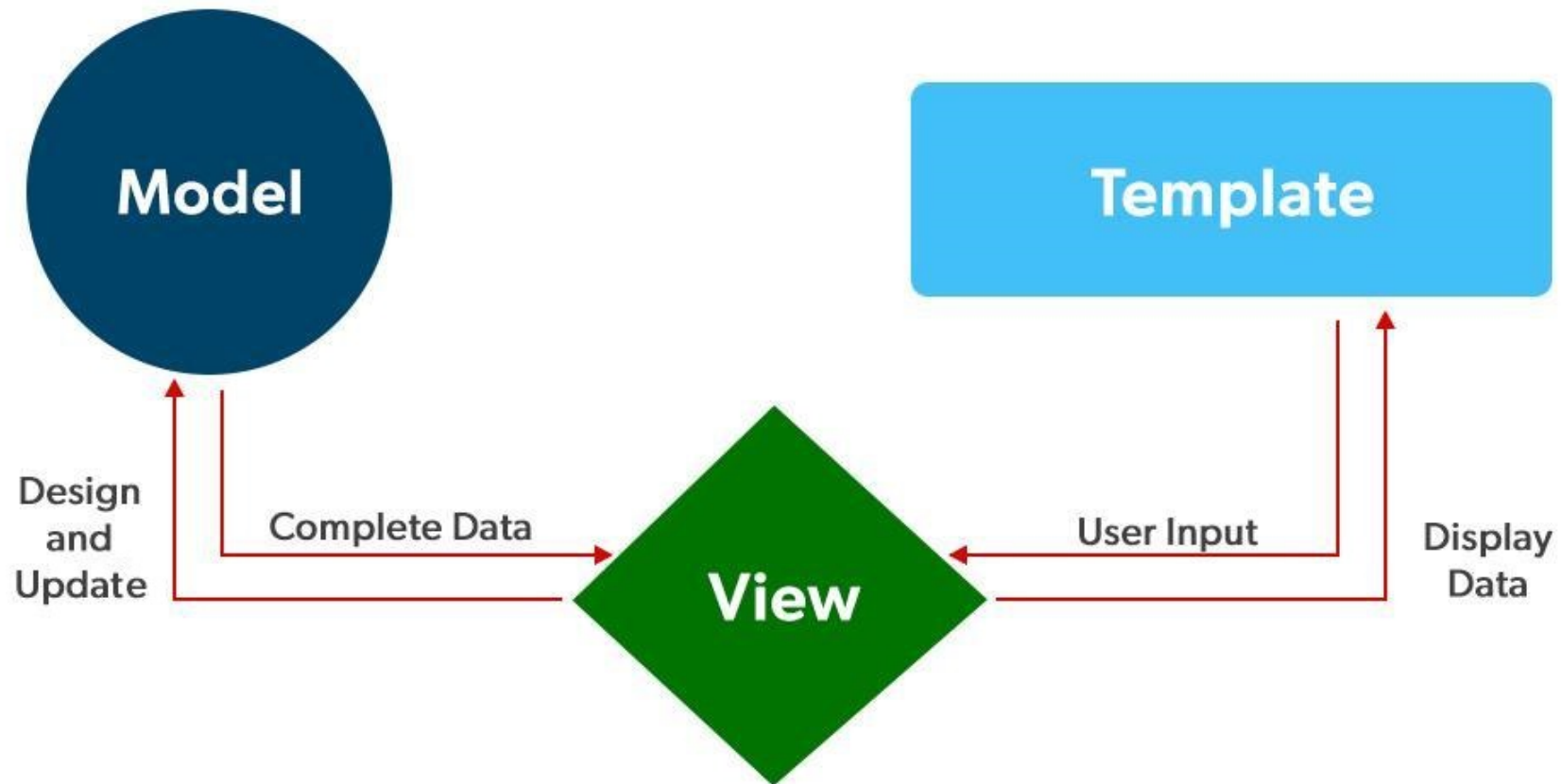
2. View

- View handles business logic and user interface rendering. It processes user requests and prepares data for display.
- Interacts with models to fetch or update data.
- Passes data to templates for rendering.
- Views are implemented as Python functions or classes that return HTTP responses.

3. Template

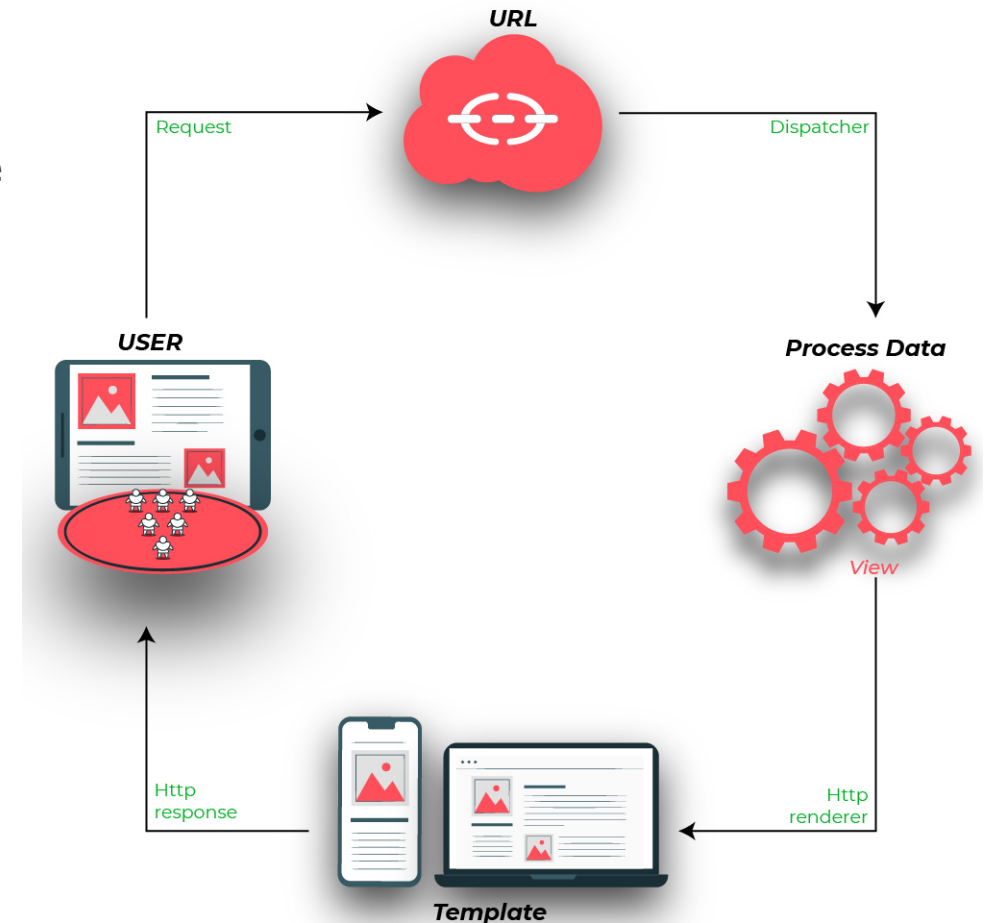
- Template is responsible for presenting data to the user. It combines static content with dynamic data using template syntax.
- Contains HTML, CSS, and JavaScript for the user interface.
- Uses Django Template Language (DTL) to insert dynamic data.
- Displays data passed from views in a structured format.

INTRODUCTION



VIEWS IN DJANGO

- In Django's **MVT architecture**, views handle user requests and return responses. They act as a **bridge between models (data) and templates (UI)**, deciding what the user sees in the browser.
- A view is a Python **function or class** that receives an HTTP request, processes data, and returns a response. This response can be:
 - An HTML page (rendered from a template)
 - A redirect (to another URL)
 - An error (like 404 Not Found)
 - JSON, XML, images, or any browser-compatible content



VIEWS IN DJANGO

- Let's use an example. Consider a project named myFirstApp, with an app named myApp. After you have the project ready, we can create a view in myApp/views.py.

```
# import HttpResponse from django
from django.http import HttpResponse
# import datetime
import datetime
```

```
# create a function-based view
def app_view(request):
    # fetch current date and time
    now = datetime.datetime.now()
    # convert to string
    html = "Time is {}".format(now)
    # return HttpResponse
    return HttpResponse(html)
```

VIEWS IN DJANGO

- Let's get this view to work, in myApp/urls.py:

```
from django.urls import path
```

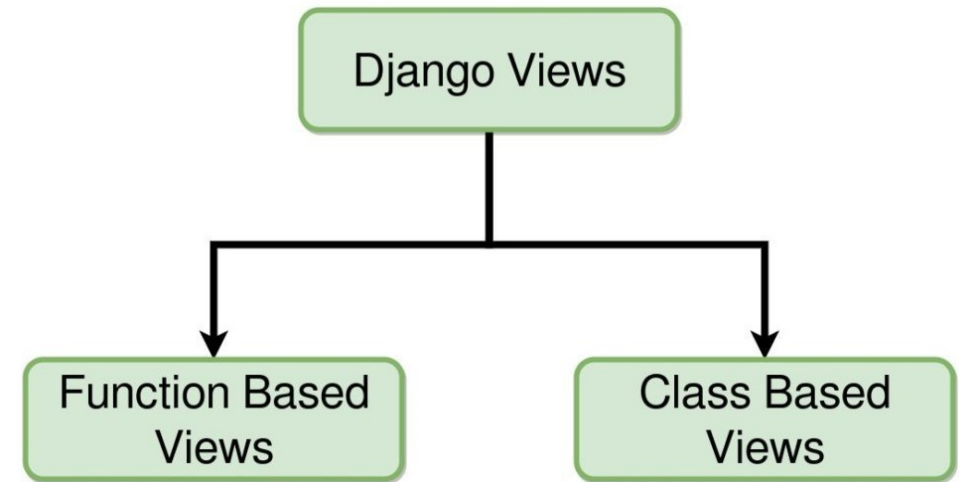
```
# importing views from views..py
```

```
from .views import example_view
```

```
urlpatterns = [  
    path('', example_view),  
]
```

VIEWS IN DJANGO

- Django provides **two main types of views** to handle web requests and return responses:
 1. **Function-Based Views (FBVs):** Written as Python functions. They are simple and ideal for small features or straightforward logic, such as displaying a list of items, handling form submissions, or showing a single detail page.
 2. **Class-Based Views (CBVs):** Written as Python classes. They offer better organization, reusability, and support for object-oriented features. They are suited for complex applications, reusable components, or features that require handling multiple HTTP methods cleanly.



FUNCTION BASED VIEWS

- Function-Based Views (FBVs) in Django are Python functions used to handle web requests and send back web responses.
 - Each FBV is linked to a specific URL and controls what happens when that URL is visited.
 - Every view function receives an `HttpRequest` object from the browser.
 - The view returns an `HttpResponse`, which can be HTML, a redirect, or an error page.
 - FBVs provide a clear and straightforward way to define application logic.

FUNCTION BASED VIEWS

- **Function-Based List View**
- Consider a project named 'myFirstApp' having an app named 'myApp'. After setting up the project and app, let's create a function-based List View to display instances of a model. First, define the model whose instances will be displayed through this view (myApp/models.py).

```
from django.db import models

# declare a new model with the name "myAppModel"
class myAppModel(models.Model):
    # fields of the model
    title = models.CharField(max_length = 200)
    description = models.TextField()
    # renames the instances of the model with their title name
    def __str__(self):
        return self.title
```


FUNCTION BASED VIEWS

- After creating this model, we need to run the two commands listed below:

```
Python manage.py makemigrations  
Python manage.py migrate
```

- Now, create instances of the model using the Django shell by running:

```
Python manage.py shell
```

- And enter the following commands:

```
from myApp.models import myAppModel  
myAppModel.objects.create(title="title1", description="description1")  
myAppModel.objects.create(title="title2", description="description2")  
myAppModel.objects.create(title="title3", description="description3")
```

FUNCTION BASED VIEWS

- Next, register the model to view it and its data in the Django admin panel.
- In **myApp/admin.py**:

```
from django.contrib import admin
from .models import myAppModel
# Register your models here.
admin.site.register(myAppModel)
```
- With the backend setup complete, verify that instances have been created by visiting:
<http://localhost:8000/admin/myApp/myAppModel/>

FUNCTION BASED VIEWS

- Create a view and template to display and handle the form. Templates are HTML files used to display data.
- In **myApp/views.py**:

```
from django.shortcuts import render
from .models import myAppModel
def list_view(request):
    # dictionary for initial data with field names as keys
    context = {}
    # add the dictionary during initialization
    context["dataset"] = myAppModel.objects.all()
    return render(request, "list_view.html", context)
```

FUNCTION BASED VIEWS

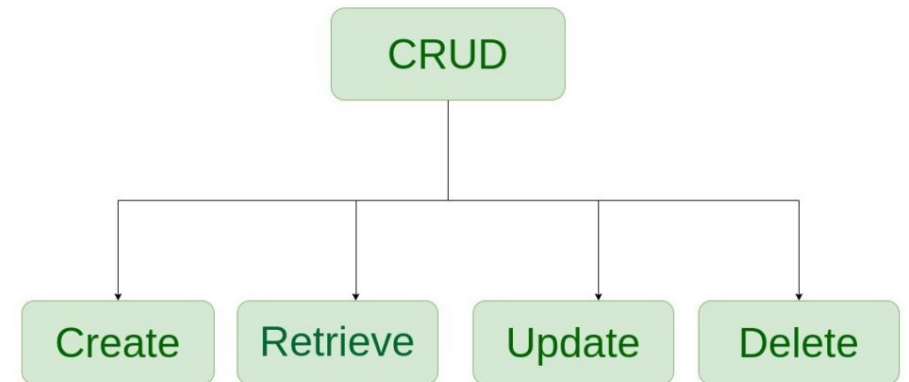
- Create a template in **myApp/templates/list_view.html**:

```
<div class="main">
    {% for data in dataset %}.
    {{ data.title }}<br/>
    {{ data.description }}<br/>
    <hr/>
    {% endfor %}
</div>
```

- Visit: **<http://localhost:8000/>**.

FUNCTION BASED VIEWS

- Function-Based Views in Django also support CRUD operations (Create, Retrieve, Update and Delete), allowing you to interact with database models to perform common web application tasks.
 - **Create:** create or add new entries in a table in the database.
 - **Retrieve:** read, retrieve, search, or view existing entries as a list (List View) or retrieve a particular entry in detail (Detail View).
 - **Update:** update or edit existing entries in a table in the database.
 - **Delete:** delete, deactivate, or remove existing entries in a table in the database.



DJANGO URLS

- The Django URL Dispatcher maps incoming HTTP requests to views, enabling precise request routing. It supports dynamic URLs, named patterns, namespaces, and class-based views for scalable, maintainable web applications.
- URL patterns define how URLs are routed to views. They are stored in the `urlpatterns` list in a Django app's `urls.py`.

- **Steps to create URL patterns:**

- Open **urls.py** in the app or project.
- Import **path()** from `django.urls`.
- Define **URL patterns** and map them to **views**.

```
from django.urls import path
from . import views
urlpatterns = [
    path('home/', views.home_view, name='home'),
    path('about/', views.about_view, name='about'),
]
```

DJANGO URLS

- **Using Regular Expression Captures in URL Patterns**
- Regex captures extract dynamic values from URLs and pass them to views as arguments.
- **Steps to use regex captures:**
 - Define a URL pattern using **re_path()** and capturing **groups ()**.
 - Add parameters in the view for each captured value.
 - Django calls the view with captured values when the URL matches.

```
from django.urls import re_path
from . import views
urlpatterns = [
    re_path(r'^blog/(?P<blog_id>\d+)/$', views.blog_detail, name='blog_detail'),
]
```

DJANGO URLS

- **Naming URL Patterns:**
- Named URL patterns allow referencing URLs without hardcoding paths in templates or Python code.
- **Steps to name URL patterns:**

- Add the name argument to the **path()** function.
- Use `{% url 'name' %}` in templates.
- Use **reverse('name')** in Python code.

```
from django.urls import path
from . import views
urlpatterns = [
    path('home/', views.home_view, name='home'),
    path('about/', views.about_view, name='about'),
]
```


DJANGO URLS

- **Inverting URL Patterns**
- Inverting URLs generates URLs from their names instead of hardcoding paths.
- **Steps to invert URL patterns:**
 - Use `{% url 'url_name' %}` in templates.
 - Use `reverse('url_name')` in Python code.
- **Example: template**
`<a href="{% url 'home' %}">Home</a>`
- **Example: Python view**

```
from django.urls import reverse  
home_url = reverse('home')
```

DJANGO URLS

- **Using Namespaces in URL Patterns**

- Namespaces organize URL patterns, avoiding naming conflicts when multiple apps exist.

- **Steps to use namespaces:**

- Include app URLs in the project `urls.py` with namespace parameter.
- Reference URLs using **'namespace:url_name'** in templates or Python code.

- **Example:** `urls.py`

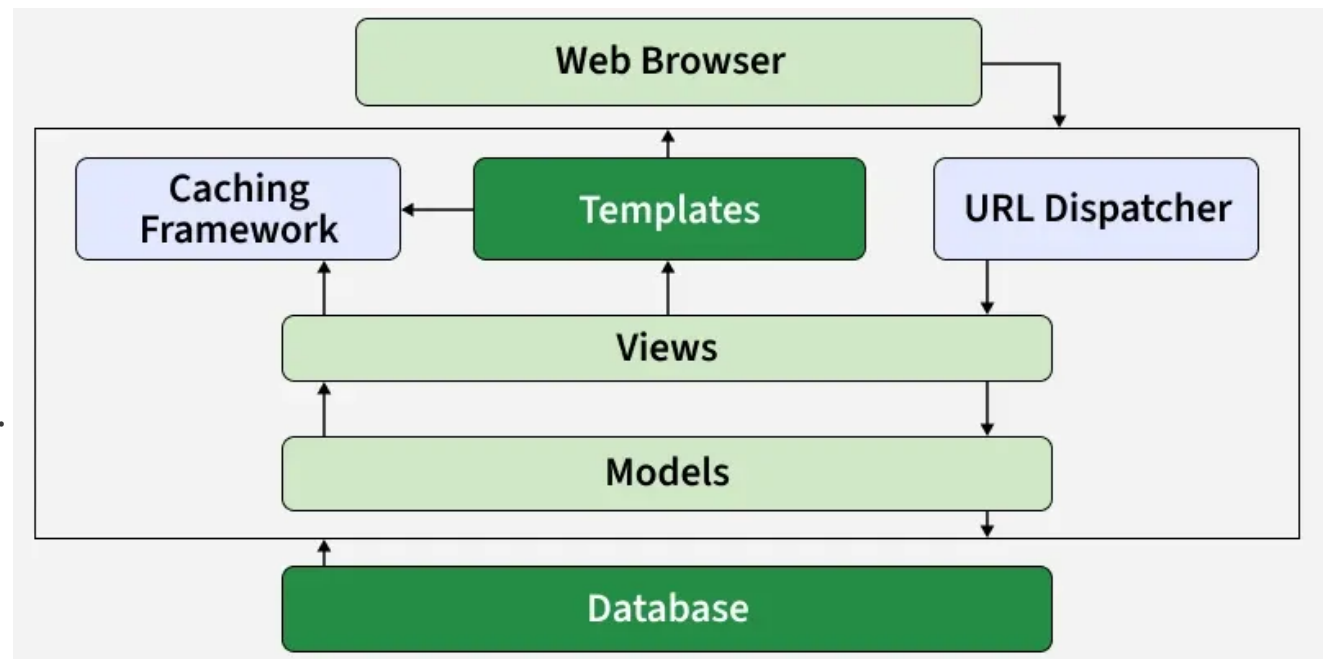
```
from django.urls import path, include
urlpatterns = [
    path('books/', include(('books.urls', 'books'), namespace='books')),
]
```

- **Example:** template reference

```
<a href="{% url 'books:book_detail' book_id=1 %}">Book Detail</a>
```

DJANGO TEMPLATES

- A Django template is an HTML file that can also include CSS and JavaScript. It is used to create dynamic web pages.
 - Renders data sent from views inside the HTML structure.
 - Helps define the layout and design of each page.
 - Supports template tags and variables for dynamic content.
 - Makes it easy to separate presentation from application logic.



DJANGO TEMPLATES

- Django provides two ways to organize templates based on your project structure:
 - **Project-level templates:** A single template directory shared across all apps.
 - **App-level templates:** Separate template directories for individual apps, useful in large projects or when different layouts are needed
- A single template directory at the project level can be used for simplicity, storing all templates in one central location.
- **Configuration in settings.py**
 - Create a folder named templates in the project's base directory (BASE_DIR).
 - Update the TEMPLATES setting in settings.py to include this folder.

DJANGO TEMPLATES

```
TEMPLATES = [  
    {  
        #By default, Django Templates (DTL) is used.  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
        'DIRS': [BASE_DIR / 'templates'],  
        'APP_DIRS': True,  
        # Options to configure template behavior  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    },  
]
```

DJANGO TEMPLATES

- Templates are not limited to displaying static HTML, they can also present dynamic data passed from views via a context dictionary.
- **Example:** Consider a project called 'exampleProject' with an app named 'example'.
- l. **views.py**
- Define three views in **example/views.py**:
 - **simple_view:** Renders static content.
 - **check_age:** Handles form input and applies conditional rendering.
 - **loop:** Passes a list and string to demonstrate looping and filters.

DJANGO TEMPLATES

```
from django.shortcuts import render

def simple_view(request):
    context = {"data": "Django is the best"}
    return render(request, "example.html",
context)

def check_age(request):
    age = None
    if request.method == 'POST':
        # request.POST.get returns a string,
default to "0"
        age = int(request.POST.get('age', 0))
    return render(request, 'check_age.html',
{'age': age})
```

```
def loop(request):
    data = "Django is the best"
    number_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    context = {
        "data": data,
        "list": number_list
    }
    return render(request, "loop.html", context)
```

DJANGO TEMPLATES

2. `urls.py`

- Define the URL patterns in your app's **`urls.py`** file to link each view to a specific route:

```
from django.urls import path
from mini import views # Assuming your app is named 'mini'
urlpatterns = [
    path('simple', views.simple_view),
    path('condition', views.check_age),
    path('loop', views.loop),
]
```


DJANGO TEMPLATES

- **Template Files**

- l. **example.html**

- Displays static data passed from the **simple_view**.

- In **app_name/templates/example.html**:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Homepage</title>
</head>
<body>
    <h1>Welcome!.</h1>
    <p>{{ data }}</p>
</body>
</html>
```

DJANGO TEMPLATES

2. check_age.html

- Accepts user input and displays different messages based on the entered age.
- In `app_name/templates/check_age.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Age Checker</title>
</head>
<body>
  <h1>Welcome to the Age Checker</h1>

  <form method="post">
    {% csrf_token %}
    <label for="age">Enter your age:</label>
    <input type="number" id="age" name="age"
required>
    <button type="submit">Check Age</button>
  </form>
```

```
{% if age is not None %}
  <p>Your age is: {{ age }}</p>
  {% if age >= 20 %}
    <p>You are an adult.</p>
  {% else %}
    <p>You are not an adult.</p>
  {% endif %}
{% endif %}
</body>
</html>
```

DJANGO TEMPLATES

3. loop.html

- Uses a for loop and if condition to display even numbers from a list.
- In **app_name/templates/loop.html**:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Even Numbers</title>
</head>
<body>
  <h1>{{ data }}</h1>
  <p>Even Numbers:</p>
```

```
    <ul>
      {% for number in list %}
        {% if number|divisibleby:2 %}
          <li>{{ number }}</li>
        {% endif %}
      {% endfor %}
    </ul>
  </body>
</html>
```

DJANGO TEMPLATES

- **Django Template Language**

- The Django Template Language (DTL) is the syntax used by Django's built-in template engine. It allows insertion of dynamic content, implementation of logic, and definition of structure directly within HTML without embedding Python code. It primarily consists of **variables**, **tags**, **filters**, and **comments**.

- I. **Variables**

- Variables are used to display dynamic data passed from views through the context dictionary.
 - Enclosed in double curly braces `{{ }}`.
 - Accesses values sent from the backend to the template.
- **Syntax:**
`{{ variable_name }}`
- **Example:** Variables are surrounded by `{{` and `}}`.
My first name is `{{ first_name }}`. My last name is `{{ last_name }}`.
- With a context of `{'first_name': 'John', 'last_name': 'Smith'}`, this template renders to:
My first name is John. My last name is Smith.

DJANGO TEMPLATES

2. Tags

- Tags add control flow and logic to templates, such as loops, conditionals, and inheritance.
- **Syntax:** `{% tag_name %}`
- **Example:** Tags are surrounded by `{% and %}` like this: `{% csrf_token %}`
- Common tags include: **if, for, block, extends, include, csrf_token, and url.**

3. Filters

- Filters are used to modify variable output before rendering on the page.
 - Applied using the pipe symbol `|`.
 - Can be chained for multiple transformations.
- **Syntax:** `{{ variable_name | filter_name }}`
- **Example:** `{{ value | length }}`
- If value is `['a', 'b', 'c', 'd']`, the output will be **4**.
- Common filters: **length, lower, title, date, default, truncatechars.**

DJANGO TEMPLATES

4. Comments

- Comments are used to hide or disable parts of a template that should not be rendered.
- **Syntax:** `{% comment 'comment_name' %}`
`{% endcomment %}`
- **Example:** `{% comment "Optional note" %}`
Commented out text with `{{ create_date|date:"c" }}`
`{% endcomment %}`

DJANGO TEMPLATES

- Django templates allow HTML to be dynamically generated by combining static content with data passed from views. Variables are used to display this dynamic data within templates. Important things about variables are:
 - Variables come from the **context dictionary** provided by the view.
 - They render values directly into HTML.
 - Can display dynamic content such as **strings, numbers, lists, or objects**.
 - Enable separation of presentation from business logic.
- **Syntax** `{{ variable_name }}`
- **Example:** Passing a context with the following data:

```
{
  'first_name': 'John',
  'last_name': 'Smith'
}
```
- And in template:

```
My first name is {{ first_name }}.
My last name is {{ last_name }}.
```
- **Output:**

```
My first name is John.
My last name is Smith.
```
- **Django Template Language (DTL)** allows dynamic data from views to be rendered directly within HTML templates.

DJANGO TEMPLATES

- **Implementation Example**

- Consider a project called 'exampleProject' with an app named 'example':

- **I. Create a View with Context Data**

- In **example/views.py**:

```
from django.shortcuts import render
def example_view(request):
    context = {
        "first_name": "John",
        "last_name": "Smith",
    }
    return render(request, "example.html", context)
```


DJANGO TEMPLATES

2. Configure URL Routing

- In **example/urls.py**:

```
from django.urls import path
from . import views
urlpatterns = [
    path('', views.example_view, name = 'example_view'),
]
```

3. Create the Template

- Create a file named **example.html** inside the app's **templates** folder. If the folder doesn't exist, create it first. All HTML files for the app should be placed in this folder.

```
My First Name is {{ first_name }}.
<br/>
My Last Name is {{ last_name }}.
```

- Run the development server using **python manage.py runserver** and Visit: **<http://127.0.0.1:8000/>**.

DJANGO TEMPLATES

- Django's template engine provides template tags that add logic and control flow to templates. Tags allow execution of operations such as conditional checks, loops, including external files, and extending base templates. Few things about template tags are that it:
 - Enable adding logic (e.g., **if**, **for**, **block**, **extends**) directly in templates.
 - Enclosed within `{% %}` delimiters.
 - Can output content, control rendering, or include external resources.
 - Often used alongside template variables (`{{ }}`) for dynamic rendering.
- **Syntax:** `{% tag_name [arguments] %}`
- **Example:** Tags are surrounded by `{% and %}` like this: `{% csrf_token %}`
- Most tags accept arguments, for example: `{% cycle 'odd' 'even' %}`

DJANGO TEMPLATES

- **Commonly Used Django Template Tags**

- 1. **Comment**

- Comment ignores everything between `{% comment %}` and `{% endcomment %}`. **Example:**

```
{% comment "This is a note" %}  
This block will not be rendered  
{% endcomment %}
```

- 2. **Extends**

- Extends are used for template inheritance. Allows a template to inherit from a base template. **Example:** Assume the following directory structure:

```
dir1/  
  template.html  
  base2.html  
my/  
  base3.html  
  base1.html
```

- In **template.html**, the following relative paths would be valid:

```
{% extends "../base2.html" %}  
{% extends "../../base1.html" %}  
{% extends "../../../my/base3.html" %}
```

DJANGO TEMPLATES

3. if, elif, else

- Controls conditional logic within the template. **Example:**

```
{% if athlete_list %}
    Number of athletes: {{ athlete_list|length }}
{% elif athlete_in_locker_room_list %}
    Athletes are in the locker room.
{% else %}
    No athletes.
{% endif %}
```

- In the above, if **athlete_list** is not empty, the number of athletes will be displayed by the `{{ athlete_list|length }}` variable.

DJANGO TEMPLATES

4. for loop

- for loops iterate over each item in a list. **Example:** display a list of athletes provided in `athlete_list`:

```
<ul>
{% for athlete in athlete_list %}
    <li>{{ athlete.name }}</li>
{% endfor %}
</ul>
```

5. Boolean Operators with if

- "if" tag evaluates a variable, and if that variable is "**true**" (i.e. exists, is not empty, and is not a false boolean value) the contents of the block are output. One can use various boolean operators with If Template tag. **Example:**

```
<ul>
{% if score > 90 and passed %}
    Excellent!
{% endif %}
</ul>
```

DJANGO TEMPLATES

6. **firstof**

- **firstof** displays the first argument variable that is not **"false"** (i.e. exists, is not empty, is not a false boolean value, and is not a zero numeric value). Outputs nothing if all the passed variables are **"false"**. **Example:**

```
{% firstof var1 var2 var3 "Default Value" %}
```

10. **now**

- **now** displays the current **date/time** using the specified format. **Example:**

```
It is currently {% now "D d M Y" %}
```

11. **url**

- **url** returns an **absolute path** reference (a URL without the domain name) matching a given view and optional parameters. This is a way to output links without violating the **DRY principle** by having to hard-code URLs in your templates. **Example:**

```
{% url 'post-detail' post.id %}
```

DJANGO MODELS

- A Django model is a Python class that represents a database table. It lets developers manage data easily using Python instead of writing SQL.
 - Django's ORM provides an easy way to connect Python code with the database.
 - Models help create, update, and delete data in a simple manner.
 - It works smoothly with the Django admin panel for managing information.
 - It supports data checks and table relationships using fields like ForeignKey and ManyToManyField.

DJANGO MODELS

- **Creating a Django Model**
- A Django project and app are needed before defining any models. After starting an app, models can be created in the file **app_name/models.py**.
- In **models.py**:

```
from django.db import models

class ExampleModel(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField()
    image = models.ImageField(upload_to='images/')
    def __str__(self):
        return self.title
```


DJANGO MODELS

- This defines a Django model called “ExampleModel” which has three fields:
 - **title:** A character field with a 200-character limit.
 - **description:** A text field for longer input.
 - **image:** An ImageField used to store the path or reference to an uploaded image file.
 - **__str__ method** is also defined to return the title of the instance of the model when the model is printed.
- This model creates a corresponding table in the database when migrations are applied. Django maps the fields defined in Django models into table fields of the database.

DJANGO MODELS

- **Applying Migrations**

- Whenever a model is created, deleted, or updated in the project's **models.py** file, the following commands must be executed:

```
python manage.py makemigrations  
python manage.py migrate
```

- The migrations system in Django generates and applies the necessary SQL commands to update your database according to the changes in your models.
 - **makemigrations** generates the SQL commands for preinstalled apps (which can be viewed in installed apps in settings.py) and your newly created app's model which you add in installed apps.
 - **migrate** executes those SQL commands in the database file.

DJANGO MODELS

- **Registering Models in Django Admin**

- To manage your model via the admin interface:

- Open admin.py in app directory.

- Register the model:

```
from django.contrib import admin
# Register your models here.
from .models import ExampleModel
admin.site.register(ExampleModel)
```

- This code does not produce any output, it simply registers the “ExampleModel” with the admin site, so that it can be managed via the Django admin interface.
- The admin panel can now be used to create, update, and delete model instances.

DJANGO MODELS

- **Model Field Types**
- Django models consist of fields that define the structure of database tables. Each field specifies:
 - The type of data it stores (text, numbers, dates, files, etc.)
 - Constraints and validations on the data
- **Common field types include:**
 - **CharField:** Stores short text (like names or titles)
 - **TextField:** Stores large amounts of text (like descriptions)
 - **IntegerField:** Stores integer values
 - **BooleanField:** Stores True/False values
 - **DateTimeField:** Stores date and time
 - **ImageField / FileField:** Stores images or files
- Fields not only define how data is stored, but also automatically handle validation and form generation when used in Django forms.

DJANGO MODELS

- **Relationship Fields**
- Relationship fields are used when models are related to each other. They allow you to represent one-to-one, one-to-many, and many-to-many relationships in your database. Common relationship fields:
 - **ForeignKey:** Many-to-one relationship.
 - **OneToOneField:** One-to-one relationship.
 - **ManyToManyField:** Many-to-many relationship.
- Relationship fields allow easy querying of related objects using Django ORM and automatically handle database constraints.

DJANGO MODELS

- **Field Options**
- Every field in Django models can have options (attributes) to control its behavior.
- Field options allow you to:
 - Set constraints (null, blank, unique)
 - Provide default values (default)
 - Customize display names (verbose_name)
 - Add help text for forms (help_text)
 - Apply validators to enforce custom rules
- These options make Django models highly flexible and ensure data integrity in the database and forms.

BASIC APP MODEL – MAKEMIGRATIONS AND MIGRATE

- Django's **Object-Relational Mapper (ORM)** simplifies database interactions by mapping Python objects to database tables. A key feature of the ORM is migrations, which enable seamless management of changes to the database schema over time.
- **Migrations in Django**
- Migrations are files that store instructions about how to modify the database schema. These files help ensure that the database is in sync with the models defined in the Django project. Whenever changes are made to models, such as adding, modifying, or deleting fields, Django uses migrations to apply these changes to the database.
- **Commands:**
 - **makemigrations:** creates migration files based on changes made to your models.
 - **migrate:** applies the generated migration files to the database.

BASIC APP MODEL – MAKEMIGRATIONS AND MIGRATE

- **Migrations in Django**
- **Step 1: Makemigrations Command**
- The makemigrations command generates migration files that describe changes to the database schema in Python code.
 - Creates new tables when new models are added.
 - Adds new fields to existing models.
 - Alters existing fields or modifies table structure.
- **How It Works:**
- When python **manage.py makemigrations** is run, Django detects changes in the models and generates migration files.
 - Migration files are created inside the migrations/ directory of the app.
 - Each file is automatically numbered (e.g., 0001_initial.py, 0002_auto_...) to represent incremental changes.
 - These files describe changes to the database schema in Python code, ready to be applied with migrate.

BASIC APP MODEL – MAKEMIGRATIONS AND MIGRATE

- **Step 2: Migrate Command**
- After creating migration files using makemigrations, the migrate command applies these changes to the database.
 - Reads all migration files in the project.
 - Performs necessary updates to the database schema.
 - Creates or modifies tables and fields as defined in the migration files.
- **How It Works:**
- The python **manage.py migrate** command applies the changes in the correct order by reading the migration files generated by makemigrations. It handles:
 - Creating tables
 - Modifying columns
 - Adding indexes
 - Performing other database-related operations

BASIC APP MODEL – MAKEMIGRATIONS AND MIGRATE

- **Creating a Basic Django App with Migrations**
- Consider a project named 'exampleProject' having an app named 'example'. After setting up the project and app, let's define a model and use migrations to update the database accordingly.

1. **In example/models.py:**

```
from django.db import models

class ExampleModel(models.Model):
    name = models.CharField(max_length=100)
    description = models.TextField()

    def __str__(self):
        return self.name
```

BASIC APP MODEL – MAKEMIGRATIONS AND MIGRATE

2. Create Migration Files

- Let's generate migration files for our new model:

```
python manage.py makemigrations
```

- Django will create a migration file (e.g., 0001_initial.py) that includes the instructions to create the ExampleModel table in the database.

3. Apply the Migrations

- To apply the migration and create the table in the database, run the following command:

```
python manage.py migrate
```

- After running makemigrations and migrate, a new table is created in the database. This can be verified in example -> migrations -> 0001_initial.py.

BASIC APP MODEL – MAKEMIGRATIONS AND MIGRATE

- After running makemigrations, Django automatically generates migration files that describe the changes made to the database schema.
- As an example, Django generates a migration file **0001_initial.py** for the **ExampleModel**, which could look like this:

<pre>from django.db import migrations, models class Migration(migrations.Migration): initial = True dependencies = [] operations = [migrations.CreateModel(name='ExampleModel',</pre>	<pre> fields=[('id', models.AutoField(auto_created=True, primary_key=True, serialize=False, verbose_name='ID')), ('name', models.CharField(max_length=100)), ('description', models.TextField()),],),]</pre>
--	---

ORM – INSERTING, UPDATING & DELETING DATA

- Django's ORM (Object Relational Mapper) makes database work easier by letting developers use Python classes instead of writing SQL.
 - Defines the database structure using Python models.
 - Connects Python objects with database tables.
 - Allows easy creation, reading, updating, and deletion of data.
 - Removes the need to write raw SQL for most operations.
- The following steps demonstrate how to define models, interact with the database, and perform common operations using Django ORM.

ORM – INSERTING, UPDATING & DELETING DATA

- **Step 1. Defining the Models**
- Let's create models for a music application with albums and songs:

```
class Album(models.Model):  
    title = models.CharField(max_length = 30)  
    artist = models.CharField(max_length = 30)  
    genre = models.CharField(max_length = 30)  
    def __str__(self):  
        return self.title  
  
class Song(models.Model):  
    name = models.CharField(max_length = 100)  
    album = models.ForeignKey(Album, on_delete = models.CASCADE)  
  
    def __str__(self):  
        return self.name
```

ORM – INSERTING, UPDATING & DELETING DATA

- **Step 2. Accessing the Django ORM via Django Shell**
- To interact with Django's ORM, we use the Django shell. Run the following command to open an interactive Python shell in the context of your Django project:
`python manage.py shell`
- we can import our models using the following command:
`>>> from music.models import Song, Album`

ORM – INSERTING, UPDATING & DELETING DATA

- **Step 3. Django ORM Queries**

- l. **Inserting**

- **Creating and Saving a Single Object:** To create and save an Album object to the database, use the following command.

```
>>> a = Album(title="Divide", artist="Ed Sheeran", genre="Pop")
>>> a.save()
```

- Similarly, to create and save a Song object associated with the previously created Album:

```
>>> s = Song(name="Castle on the Hill", album=a)
>>> s.save()
```

- **Inserting Multiple Records:** Let's add a few more albums to the database.

```
>>> a1 = Album(title="Abbey Road", artist="The Beatles", genre="Rock")
>>> a1.save()

>>> a2 = Album(title="Revolver", artist="The Beatles", genre="Rock")
>>> a2.save()
```

- Here, we've added two more albums, both by **The Beatles**. Each call to `save()` writes the object to the database.

ORM – INSERTING, UPDATING & DELETING DATA

2. Retrieving

- **Retrieving All Records:** To retrieve all records from the Album model, we use the `all()` method.

```
>>> Album.objects.all()
<QuerySet [<Album: Divide>, <Album: Abbey Road>, <Album: Revolver>]>
```

- The **QuerySet** is a list-like object containing all the **Album** instances. The `__str__()` method is used to print the album title.
- **Filtering Records:** You can filter records based on specific conditions using the `filter()` method.

```
>>> Album.objects.filter(artist="The Beatles")
<QuerySet [<Album: Abbey Road>, <Album: Revolver>]>
```

- In this example, we retrieve albums by **The Beatles**.
- **Excluding Records:** The `exclude()` method allows you to retrieve all records except those that match a condition.

```
>>> Album.objects.exclude(genre="Rock")
<QuerySet [<Album: Divide>]>
```

- Here, we exclude albums with the genre Rock and retrieve the remaining albums.
- **Retrieving a Single Record:** To retrieve a single object based on a unique field, use `get()`. Note that this will raise an exception if multiple objects match.

```
>>> Album.objects.get(pk=3)
<Album: Revolver>
```

- Here, we're retrieving the album with primary key (pk) 3, which is **Revolver**.

ORM – INSERTING, UPDATING & DELETING DATA

3. Updating

- **Modifying an Existing Object:** To update an existing record, first retrieve the object, modify its attributes, and then save it.

```
>>> a = Album.objects.get(pk=3)
>>> a.genre = "Pop"
>>> a.save()
```

- This changes the genre of the album Revolver to 'Pop' and saves the update to the database.
- **Bulk Update:** You can also update multiple objects at once using `update()`.

```
>>> Album.objects.filter(artist="The Beatles").update(genre="Classic Rock")
```
- This will update the genre of all albums by **The Beatles** to '**Classic Rock**'.

ORM – INSERTING, UPDATING & DELETING DATA

4. Deleting

- **Deleting a Single Record:** To delete a specific record, first retrieve the object and then call the `delete()` method.

```
>>> a = Album.objects.get(pk=2)
>>> a.delete()
```

- After running this, the album with `pk=2` (e.g., Abbey Road) will be deleted from the database.
- **Deleting Multiple Records:** To delete multiple records at once, use `filter()` or `exclude()` combined with the `delete()` method.

```
>>> Album.objects.filter(genre="Pop").delete()
```

- This will delete all albums with the genre 'Pop'. Be cautious as this operation cannot be undone.

ORM – INSERTING, UPDATING & DELETING DATA

■ Common Django ORM Query Methods

- **.all():** Retrieve all objects of a model.
- **.filter():** Retrieve objects that match specific conditions.
- **.exclude():** Retrieve objects that do not match specific conditions.
- **.get():** Retrieve a single object that matches the condition (raises an error if multiple objects match).
- **.update():** Update multiple objects at once.
- **.delete():** Delete records from the database

DJANGO ADMIN INTERFACE

- The Django Admin Interface is a built-in feature that provides a web-based dashboard to manage project data through models. It enables developers and administrators to handle data efficiently without writing SQL or extra backend code.
- Key Features:
 - Provides a ready-to-use, browser-based management interface.
 - Allows adding, editing, and deleting records across registered models.
 - Supports managing users, groups, and permissions from a single place.
 - Comes preinstalled as part of `django.contrib.admin`.
 - Can be customized and extended according to project requirements.

DJANGO ADMIN INTERFACE

- **Accessing Admin Panel**

- To access the admin interface:

1. Start the development server

```
python manage.py runserver
```

2. Open the admin URL

```
http://127.0.0.1:8000/admin/
```

3. Log in using valid admin credentials.

- There are basically two ways to access the Django admin panel:

- **Superuser:** Has full access to all models and settings.

- **Staff User with Permissions:** Has limited access based on assigned permissions. Can be marked as staff (`is_staff=True`) and given model-level permissions through the admin interface or Django shell.

DJANGO ADMIN INTERFACE

- **Accessing with Superuser**
- The **admin app(django.contrib.admin)** is enabled by **default** and already added to the **INSTALLED_APPS** list present in the **settings.py** file. If you don't have a user yet, the quickest way to log in is by creating a superuser.
 1. Run the following command in terminal:
`python manage.py createsuperuser`
 2. Provide the required information:
 - Username
 - Email address (optional)
 - Password
 3. Then start the development server:
`python manage.py runserver`
- Go to **`http://127.0.0.1:8000/admin/`**, log in with your superuser credentials, and you'll see the Django Admin Dashboard. From here, we can manage all registered **models**, **users**, and **site data**.

DJANGO ADMIN INTERFACE

- By default, only Django's built-in models appear in the admin panel. To manage our own app models via the admin interface, we need to register them. Before proceeding further, ensure that the Django project and app is already created:

```
python manage.py createapp myapp
```

- **Step 1: Define a Model in models.py**

```
from django.db import models

class FacultyDetails(models.Model):
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)
    department = models.CharField(max_length=50)

    def __str__(self):
        return f"{self.first_name} {self.last_name}"
```


DJANGO ADMIN INTERFACE

- **Step 2: Register the Model in admin.py**

```
from django.contrib import admin
from .models import FacultyDetails
admin.site.register(FacultyDetails)
```

- **Step 3: Apply Migrations**

```
python manage.py makemigrations
python manage.py migrate
```

- Once registered and migrated, our model will appear on the Django admin dashboard, ready for data manipulation. Now the admin page will have the model FacultyDetails but make sure the server is up and running.

CRUD OPERATION IN DJANGO ADMIN

- Django Admin is a built-in tool that automatically generates a web interface for models, removing the need to create separate pages or forms. It provides an intuitive way to manage project data and perform CRUD(Create, Read, Update, and Delete) operations efficiently.
- Let's understand step by step how to perform CRUD operations.
- Consider a project named 'projectName' having an app named 'app'. After creating the app, make sure to add your app to the `INSTALLED_APPS` list in `settings.py` file of 'projectName'.

CRUD OPERATION IN DJANGO ADMIN

- **Step 1: Define the Book Model**

- In app/models.py, create a "Book" model to represent books in your library. This model will include common book details like title, author, published date, ISBN, number of pages, and availability status.

```
from django.db import models
```

```
class Book(models.Model):
```

```
    title = models.CharField(max_length=200)
```

```
    author = models.CharField(max_length=100)
```

```
    published_date = models.DateField()
```

```
    isbn = models.CharField(max_length=13, unique=True)
```

```
    pages = models.IntegerField()
```

```
    available = models.BooleanField(default=True)
```

```
    def __str__(self):
```

```
        return f"{self.title} by {self.author}"
```

CRUD OPERATION IN DJANGO ADMIN

- **Step 2: Migrate Your Database**

- After creating this model, we need to run two commands in order to create Database for the same:

```
python manage.py makemigrations  
python manage.py migrate
```

- **Step 3: Register the Model in Django Admin**

- To manage Book records via the admin panel, register model in app/admin.py:

```
from django.contrib import admin  
from .models import Book  
@admin.register(Book)  
class BookAdmin(admin.ModelAdmin):  
    list_display = ('title', 'author', 'published_date', 'isbn', 'available')  
    search_fields = ('title', 'author', 'isbn')  
    list_filter = ('available', 'published_date')
```

- This customizes the admin interface to show important book details and helps to find books faster using search and filters.

CRUD OPERATION IN DJANGO ADMIN

- **Step 4: Create a Superuser to Access Admin**

- To create a superuser, run following command:

```
python manage.py createsuperuser
```

- Provide a username, email, and password. This superuser account has full access to the admin panel. After creation, log in using these credentials to access and manage your models.

- **Step 5: Run the Development Server**

- Start the Django server:

```
python manage.py runserver
```

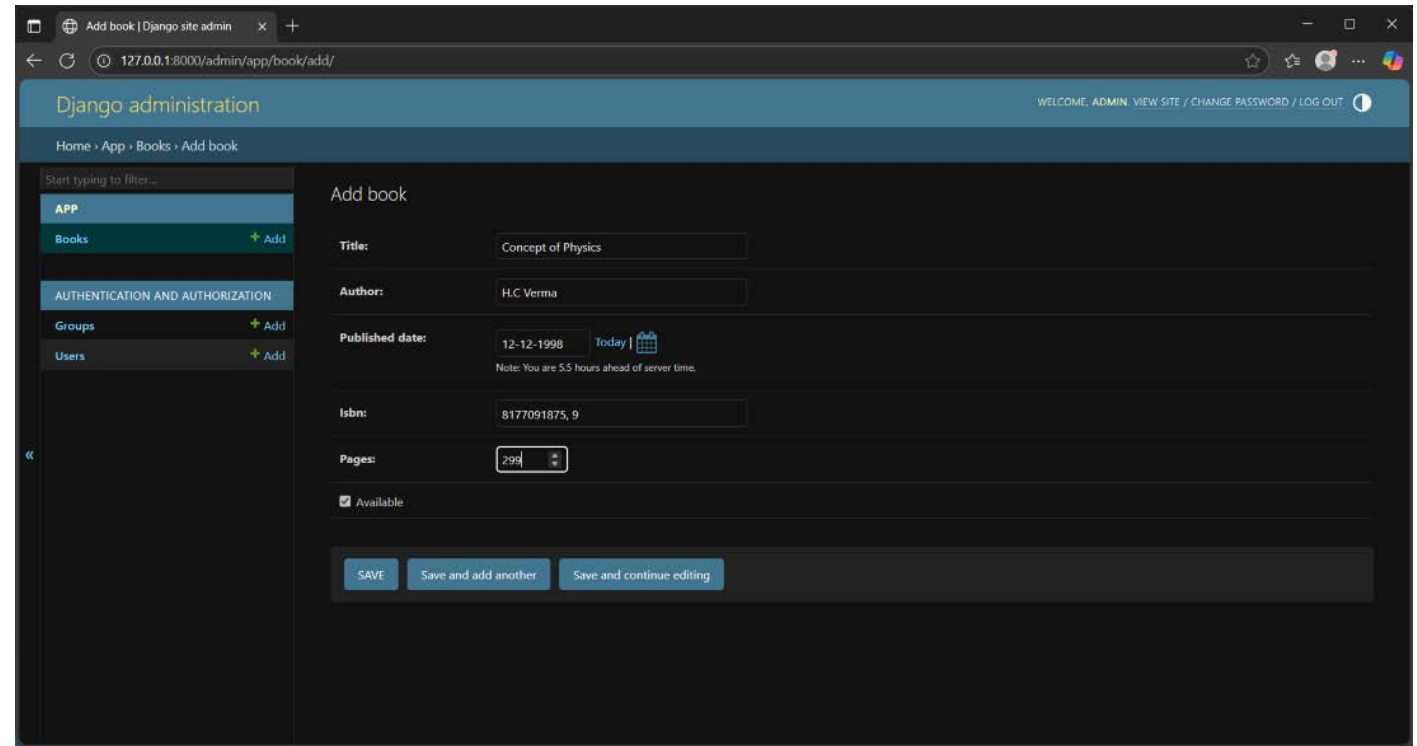
- Visit: **<http://127.0.0.1:8000/admin/>** and log in with your superuser credentials to access the admin panel.

CRUD OPERATION IN DJANGO ADMIN

■ Performing CRUD Operations

I. Create New Book Records

- Click Books in the admin sidebar.
- Click Add Book.
- Fill in details like title, author, and ISBN.
- Click Save to add the book.



The screenshot shows the Django Admin interface for adding a new book. The browser address bar shows the URL `127.0.0.1:8000/admin/app/book/add/`. The page title is "Django administration" and the user is logged in as "ADMIN". The sidebar on the left shows the "Books" link highlighted under the "APP" section. The main form is titled "Add book" and contains the following fields:

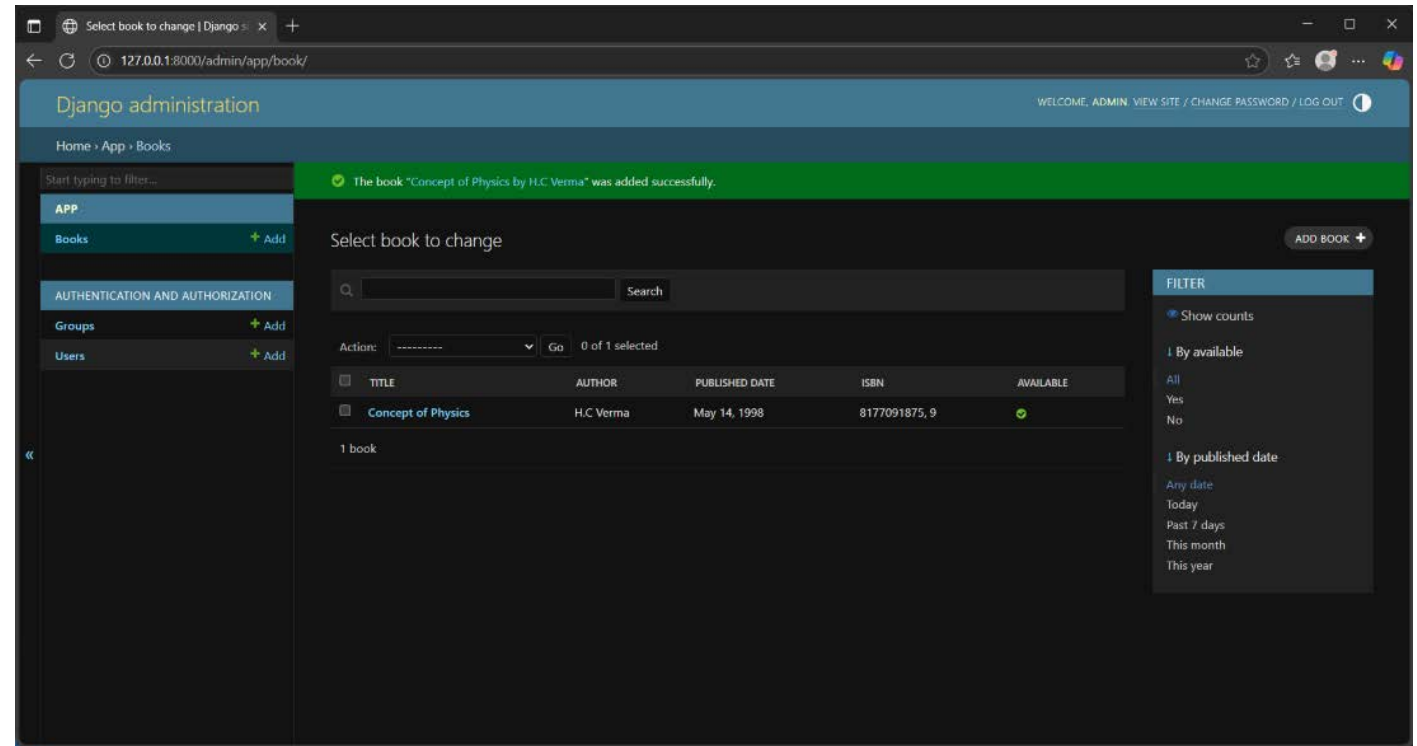
- Title:** Concept of Physics
- Author:** H.C Verma
- Published date:** 12-12-1998 (with a calendar icon and a note: "Note: You are 5.5 hours ahead of server time.")
- Isbn:** 8177091875, 9
- Pages:** 294 (with a spinner icon)
- Available:** ☒

At the bottom of the form, there are three buttons: "SAVE", "Save and add another", and "Save and continue editing".

CRUD OPERATION IN DJANGO ADMIN

2. View (Read) Books

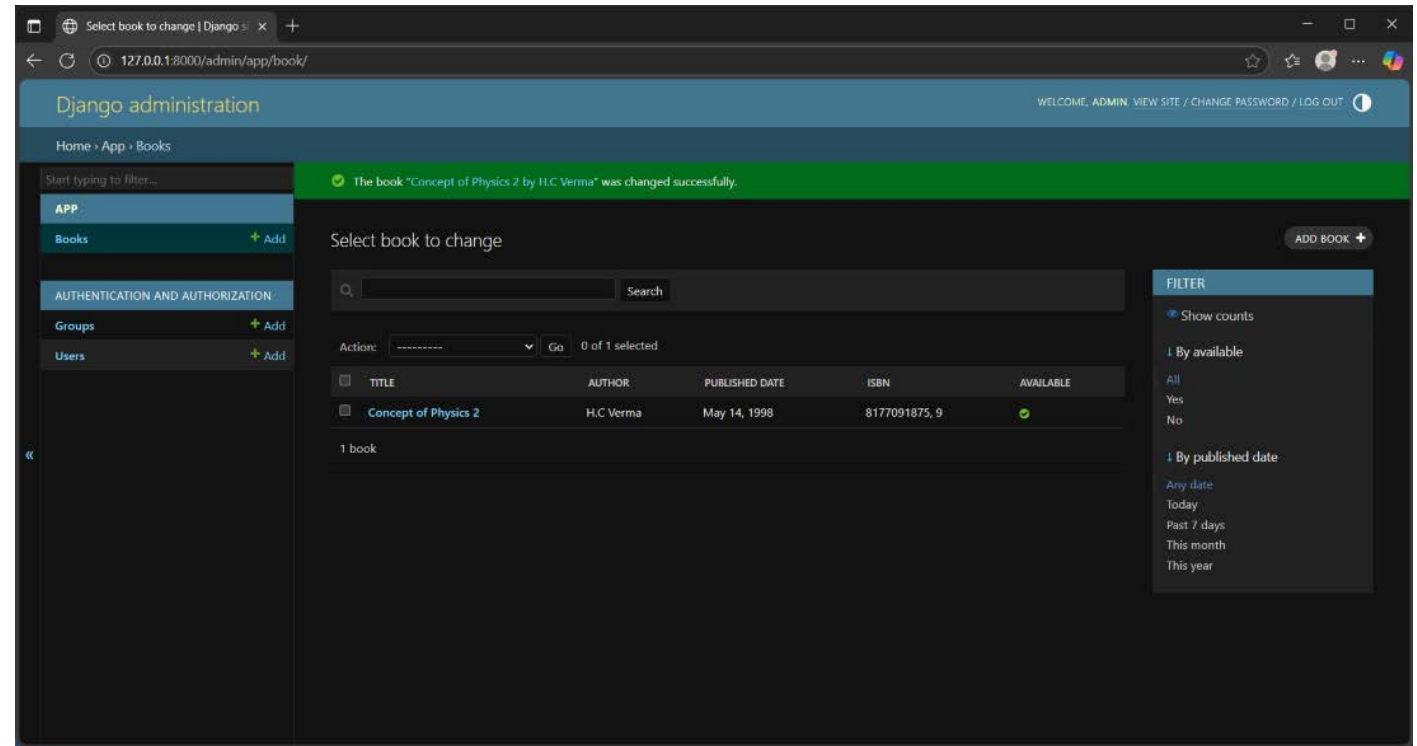
- The Books list page shows all saved books.
- Use the search bar or filters on the right to find specific books quickly.



CRUD OPERATION IN DJANGO ADMIN

3. Edit Existing Books

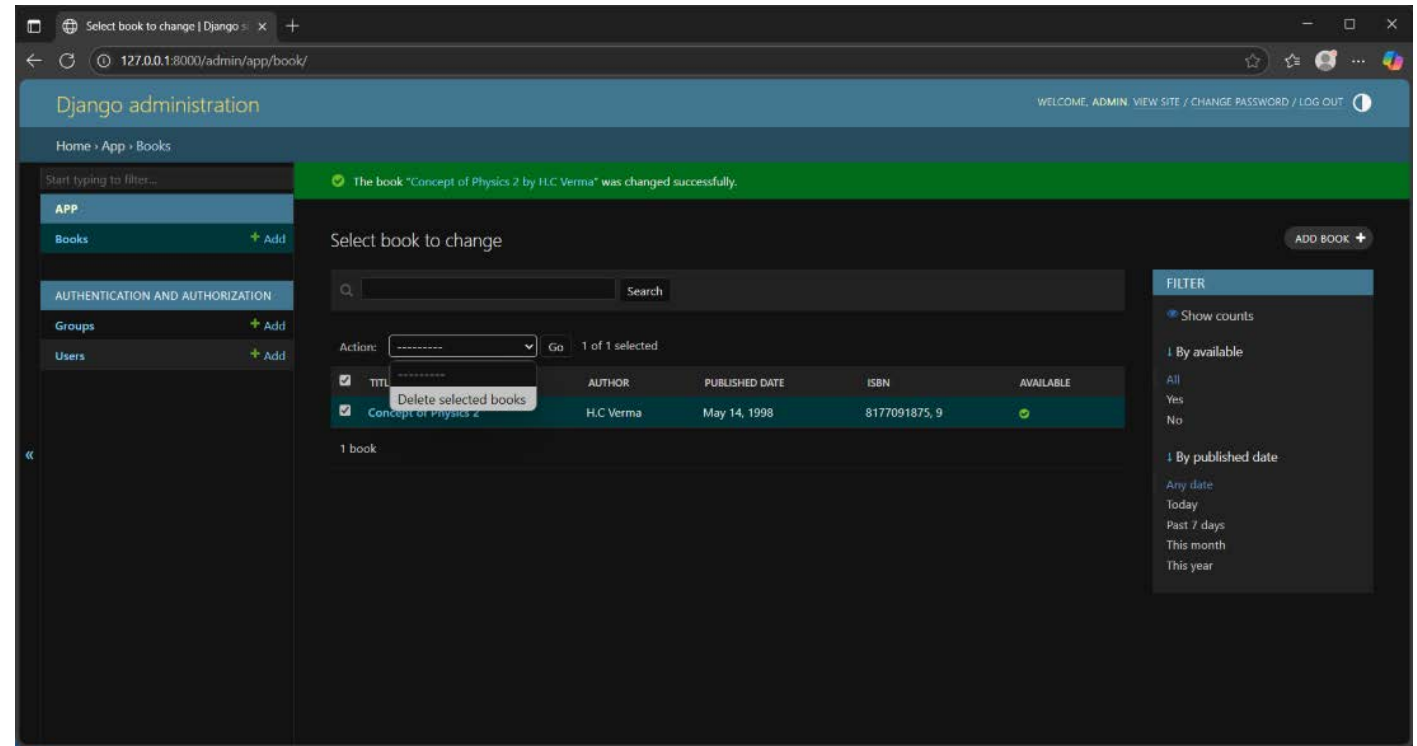
- Click on a book title from the list.
- Update any details in the form.
- Click Save to apply changes.



CRUD OPERATION IN DJANGO ADMIN

4. Delete Books

- Select the book/books.
- Select the DELETE option from the dropdown above the list of books.
- Confirm to permanently remove the book.



DJANGO QUERYSETS: INTRODUCTION AND USING GET()

- A `QuerySet` in Django is a collection of database records that works like a Python list but offers more features for handling data efficiently.
 - Allows filtering, sorting, and organizing records before they are fetched from the database.
 - Uses lazy loading, so data is retrieved only when the `QuerySet` is used.
 - Supports chaining multiple filters to get precise results.
 - Fetches only the required data, which improves performance.
- **`get()` Method**
- The **`get()`** method returns exactly one object that matches the conditions provided. It has specific behaviors and use cases as described below:
 - Raises a `DoesNotExist` exception if no object is found.
 - Raises a `MultipleObjectsReturned` exception if more than one object matches.
 - Ideal when there is only one record for the given lookup.
 - Unlike `filter()`, `get()` returns a single model instance, not a `QuerySet`.

DJANGO QUERYSETS: INTRODUCTION AND USING GET()

- **Syntax:** `Model.objects.get(**kwargs)`
 - **Model** is a Django model.
 - **kwargs** are the conditions used to find the object.
- Consider a **Book** model with fields like **title** and **author**. To get the book titled "**The Great Gatsby**":

```
from app.models import Book
```

```
try:
```

```
    book = Book.objects.get(title='The Great Gatsby')
```

```
    # Use the book object here
```

```
except Book.DoesNotExist:
```

```
    print("No book found with that title.")
```

```
except Book.MultipleObjectsReturned:
```

```
    print("Multiple books found with that title.")
```

DJANGO QUERYSETS: INTRODUCTION AND USING GET()

- **Using Multiple Conditions (AND)**

- To use multiple conditions with the `get()` method in Django:

try:

```
book = Book.objects.get(title='The Great Gatsby', author='F. Scott Fitzgerald')
```

except `Book.DoesNotExist`:

```
print("No matching book found.")
```

except `Book.MultipleObjectsReturned`:

```
print("More than one book matches your query.")
```

DJANGO QUERYSETS: INTRODUCTION AND USING GET()

- **Using Primary Key with get()**

- Fetching objects by their primary key (usually id) is a common and efficient way to retrieve a single record:

```
book = Book.objects.get(pk=1) # Get book with primary key 1
```

- **pk** is a shortcut for the model's primary key field.
- This query retrieves the exact object with **id=1**.

- **Practical Uses of get()**

- Authenticating users by **username** or **email**.
- Retrieving records using **unique fields** like primary keys.
- Getting **exact matches** when only one record is expected.

DJANGO QUERYSETS: INTRODUCTION AND USING GET()

- **Using get() in the Django Shell**
- The **Django shell** is an interactive Python environment for experimenting with models and QuerySets in real time. It's perfect for:
 - Testing database queries
 - Exploring data
 - Debugging without writing views or templates
- **Example: Fetching a Book Object Using get()**
- Consider the **book** model. Data can be fetched from it using the Python shell:
 1. **Activate the Shell:**

```
python manage.py shell
```
 2. **Use the following commands** (this should print the details of book with **id = 2**):

```
>>> from app.models import Book
>>> book = Book.objects.get(id=2)
>>> print(book)
```

DJANGO QUERYSETS FILTER()

- QuerySet filtering helps retrieve only the required records from the database, making data handling fast and efficient.
 - filter() and exclude() select or remove records based on given conditions.
 - Lookup expressions like exact, icontains, lte, and gte refine query accuracy.
 - QuerySets are lazy and support chaining multiple conditions for precise results.
 - Q objects enable advanced filtering using OR, AND, and NOT operations.
- **QuerySet Filtering Implementation**
- Consider a project named 'projectApp' with a model 'User'. Filtering datasets using Django's filter(), exclude(), and advanced Q objects with a model named User.

DJANGO QUERYSETS FILTER()

- **Step 1: Define the User Model**

- In projectApp/models.py, create a User model with fields for **user_name**, **city**, and **country**:

```
from django.db import models
```

```
class User(models.Model):
```

```
    user_name = models.CharField(max_length=20)
```

```
    city = models.CharField(max_length=20, blank=True, null=True)
```

```
    country = models.CharField(max_length=20, blank=True, null=True)
```

```
    def __str__(self):
```

```
        return self.user_name
```


DJANGO QUERYSETS FILTER()

- **Step 2: Register the Model in Django Admin**
- To make the User model accessible in the Django Admin interface, register it in **projectApp/admin.py**:

```
from django.contrib import admin
from .models import User
admin.site.register(User)
```
- To access the User model through the Django admin panel, create a superuser:

```
python manage.py createsuperuser
```

DJANGO QUERYSETS FILTER()

- **QuerySet Filtering Examples**

- With the User model in place, it's time to explore how to filter data using QuerySets.

- 1. **Filtering All Users**

- To retrieve all users from a specific country, use **filter()**:
`users = User.objects.filter(country='Germany')`
- This retrieves all users where the country is 'Germany'.

- 2. **Excluding Specific Records**

- To **exclude** records that match a condition, use **exclude()**:
`users = User.objects.filter(country='Germany').exclude(city='Berlin')`
- This retrieves all users from Germany, excluding those from Berlin.

- 3. **Using Q Objects for Advanced Filtering**

- **Q** objects allow for more complex queries, like **OR conditions**:
`from django.db.models import Q`
`users = User.objects.filter(Q(country='Germany') | Q(city='New York'))`
- This returns users who are from either Germany or New York.

DJANGO QUERYSETS ORDER_BY()

- The `order_by()` method in Django QuerySets is used to sort results based on one or more fields.
 - Supports sorting in both ascending and descending order.
 - Can sort by fields like name, date, salary, or any model field.
 - Sorting happens at the database level for better performance.
 - Helps display data in a structured and meaningful order.

DJANGO QUERYSETS ORDER_BY()

- Consider a project named 'projectApp' with a model 'EmployeeDetails'.

- In **models.py**:

```
from django.db import models

class EmployeeDetails(models.Model):
    EmployeeId = models.AutoField(primary_key=True)
    EmployeeName = models.CharField(max_length=20)
    EmployeeDepartment = models.CharField(max_length=20, blank=True, null=True)
    Country = models.CharField(max_length=20, blank=True, null=True)
    Salary = models.IntegerField(blank=True)
    def __str__(self):
        return self.EmployeeName
```

DJANGO QUERYSETS ORDER_BY()

- After creating this model, run the following two commands to **create and apply migrations**, which will create the corresponding database tables.

```
python manage.py makemigrations  
python manage.py migrate
```

- Add some sample data via the Django admin or shell before testing.

DJANGO QUERYSETS ORDER_BY()

- Open the Django shell:

```
python manage.py shell
```

- l. **Sort Employees in Ascending Order by Salary**

```
from yourapp.models import EmployeeDetails  
# Retrieve all employees ordered by Salary (lowest first)  
employees = EmployeeDetails.objects.all().order_by('Salary')  
for emp in employees:  
    print(emp.EmployeeName, emp.Salary)
```

- This will display employees starting from the lowest salary.

DJANGO QUERYSETS ORDER_BY()

2. Sort Employees by Salary in Descending Order

```
# Using '-' prefix to sort descending by Salary
employees = EmployeeDetails.objects.all().order_by('-Salary')
for emp in employees:
    print(emp.EmployeeName, emp.Salary)
```

- The '-' **before** the field name **reverses the order**, so the highest salaries appear first.

3. Sort by Multiple Fields

```
# Order first by Salary ascending, then by EmployeeName ascending
employees = EmployeeDetails.objects.all().order_by('Salary', 'EmployeeName')
for emp in employees:
    print(emp.EmployeeName, emp.Salary)
```

- Employees are sorted by salary, when salaries are equal, they are sorted alphabetically by name.